

深度学习作业报告

— 循环神经网络 (RNN) 与唐诗生成

姓名：李睿 学号：2351610

2026 年 3 月 21 日

1 模型原理：RNN、LSTM 与 GRU

1.1 循环神经网络 (RNN)

循环神经网络是一类专门处理序列数据的神经网络结构。与前馈神经网络不同的地方是，RNN 在每个时间步 t 不仅接受当前输入 x_t ，还接收来自上一时刻的隐藏状态 h_{t-1} ，从而在网络内部保留“记忆”，对历史信息进行建模。在本次作业中，我也将对 RNN 代码进行补全，从而实现唐诗的生成。下面我将首先介绍一下和 RNN 相关的基础知识。

1.1.1 前向传播公式

RNN 的核心递推公式为：

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h) \quad (1)$$

$$\hat{y}_t = \text{softmax}(W_{hy} h_t + b_y) \quad (2)$$

其中 $h_t \in \mathbb{R}^{d_h}$ 是隐藏状态， $x_t \in \mathbb{R}^{d_x}$ 是当前时刻的输入， W_{hh} 、 W_{xh} 、 W_{hy} 分别是对应的权重矩阵， b_h 、 b_y 是偏置向量。

1.1.2 RNN 的局限性

标准 RNN 在处理长序列时存在严重的梯度消失和梯度爆炸问题。当序列长度较长时，误差信号沿时间反向传播会呈指数级衰减，导致模型难以学习到长距离的依赖关系。

1.2 长短时记忆网络 (LSTM)

长短时记忆网络由 Hochreiter & Schmidhuber (1997) 提出，通过引入三个“门”机制（遗忘门、输入门、输出门）和独立的细胞状态 c_t 来有效缓解梯度消失问题，从而捕获长距离依赖。

1.2.1 门控机制

LSTM 在每个时间步计算如下：

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{遗忘门}) \quad (3)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{输入门}) \quad (4)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{候选细胞状态}) \quad (5)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{细胞状态更新}) \quad (6)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{输出门}) \quad (7)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{隐藏状态输出}) \quad (8)$$

其中 $\sigma(\cdot)$ 为 Sigmoid 激活函数, $\odot$ 表示逐元素乘法。

1.2.2 各门的作用

- **遗忘门 f_t** : 决定从上一时刻细胞状态 c_{t-1} 中“遗忘”多少信息, 输出在 $(0, 1)$ 之间, 越接近 0 则遗忘越多。
- **输入门 i_t** : 决定将当前候选信息 $\tilde{c}_t$ 中的多少写入细胞状态。
- **输出门 o_t** : 决定将细胞状态中的多少信息暴露给外部隐藏状态 h_t , 进而影响下一时刻的计算和当前时刻的输出。

1.3 门控循环单元 (GRU)

门控循环单元 (Gated Recurrent Unit, GRU) 由 Cho 等人 (2014) 提出, 是对 LSTM 的简化改进。GRU 将遗忘门和输入门合并为一个**更新门** z_t , 并引入**重置门** r_t , 去掉了独立的细胞状态 c_t , 参数更少, 训练速度更快, 在很多任务上与 LSTM 性能相当。

2 诗歌生成过程

本实验采用基于字符级 LSTM 的语言模型来实现唐诗自动生成。整体流程分为**数据预处理**、**模型构建**、**模型训练**和**诗歌生成**四个阶段。

2.1 数据预处理

原始数据格式 数据集 `poems.txt` 中每行格式为” 标题: 正文”, 例如:

首春: 寒随穷律变, 春逐鸟声开。初风飘带柳, 晚雪间花梅。

处理步骤

1. **解析内容**: 按冒号分割标题与正文, 保留正文部分。
2. **过滤**: 去除含有特殊符号(下划线、括号、书名号等)的诗句; 过滤长度小于 5 或大于 80 的诗歌, 确保批次对齐效率。
3. **添加特殊标记**: 在每首诗的首尾分别添加开始标记 G (start_token) 和结束标记 E (end_token), 格式变为 $G[\text{诗歌内容}]E$ 。
4. **构建词表**: 统计所有字符出现频次, 按频次降序排列, 构建字符到整数的映射字典 `word_int_map` 及其逆映射 `vocabularies`。
5. **向量化**: 将每首诗从字符序列转换为整数序列(词表索引), 得到 `poems_vector`。
6. **按长度排序**: 对所有诗歌按字符数从小到大排序, 以便 batch 内序列长度相近, 减少无效 padding。

批数据生成 函数 `generate_batch` 将数据切分为若干大小为 `BATCH_SIZE` 的批次。对于每个批次, 输入 X 为诗歌序列去掉最后一个字符, 标签 Y 为同一序列向左移一位(即 $Y_t = X_{t+1}$), 形成标准的**语言模型自回归训练对**。

2.2 模型结构

本实验模型由三个模块组成, 包括以下内容:

1. **词嵌入层 (Embedding)**: 将字符索引映射到 100 维稠密向量, 初始化采用 $[-1, 1]$ 均匀分布的随机权重, 在训练中与网络一同优化。
2. **双层 LSTM**: 隐藏维度为 128, 设置 `batch_first=True`, 输入维度为嵌入维度 (100), 输出为每个时间步的隐藏状态序列 (维度 128)。
3. **全连接层 + 激活**: 将 LSTM 输出的 128 维向量映射到词表大小的 logits 向量, 经 ReLU 激活后接 LogSoftmax 得到对数概率, 配合 `NLLLoss` 计算负对数似然损失。

2.3 模型训练

损失函数与优化器 损失函数采用负对数似然损失, 即最大化模型对训练序列中每个字符的预测概率:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(y_t | y_{<t}) \quad (9)$$

优化器采用 RMSprop，学习率设置为 0.01。在每个 batch 更新后，使用梯度裁剪 (clip_grad_norm_, 阈值为 1) 防止梯度爆炸。

2.4 诗歌生成

生成时采用**贪心解码** (Greedy Decoding)，步骤如下：

1. 将指定的起始字符（如”日”）作为第一个输入，转换为对应的整数索引。
2. 将当前已生成的诗歌序列（从起始字符到最新字符）整体输入模型，获得最后一个时间步的输出概率分布（通过 is_test=True 仅取最后一步输出）。
3. 取概率最大的字符 (argmax) 作为下一个生成字符，追加到诗歌字符串。
4. 重复步骤 2-3，直到生成结束标记 E 或达到最大长度限制（50 字）。
5. 去掉开始/结束标记，格式化输出。

该过程每次以之前生成的前缀为条件，预测下一个最可能的字符，逐字构成完整诗歌。

3 实验结果

3.1 训练过程截图

下图为训练过程中，第一个回合的训练输出结果，从中我们可以看到预测序列与真实标签对比输出。我发现模型在训练初期随机预测，经若干轮训练后逐渐向目标序列靠拢：

```
*****
epoch   0 batch number 134 loss is:  6.160313606262207
prediction [66, 44, 33, 6, 0, 0, 33, 8, 43, 5, 1, 1, 6, 6, 4, 6, 29, 0, 6, 6, 4, 5, 1, 1, 3, 27, 9, 6, 0, 0, 15, 6, 43, 5, 1, 1, 3, 6, 9, 6, 0, 0, 15, 6, 4, 5, 1, 1, 3, 3]
b_y       [691, 34, 601, 93, 182, 0, 68, 87, 233, 177, 84, 1, 163, 23, 110, 825, 2824, 0, 9, 1428, 422, 570, 1474, 1, 738, 173, 879, 863, 1344, 0, 644, 2139, 181, 1555, 841, 1, 37
4, 62, 1115, 193, 939, 0, 195, 484, 39, 23, 1193, 1, 3, 3]
*****
epoch   0 batch number 135 loss is:  6.087086200714111
prediction [66, 6, 33, 22, 0, 0, 22, 5, 4, 26, 5, 1, 8, 5, 4, 26, 0, 0, 40, 5, 4, 26, 5, 1, 3, 30, 4, 26, 29, 0, 40, 5, 4, 5, 5, 1, 3, 8, 4, 16, 0, 0, 40, 41, 4, 26, 5, 1, 3, 3]
b_y       [565, 715, 282, 74, 161, 0, 163, 1182, 449, 504, 353, 1, 105, 98, 429, 12, 94, 0, 1431, 96, 255, 26, 546, 1, 146, 59, 273, 57, 0, 0, 351, 189, 93, 46, 214, 1, 68, 269, 4
, 73, 366, 0, 205, 450, 92, 1323, 103, 1, 3, 3]
*****
epoch   0 batch number 136 loss is:  6.036292076110884
prediction [82, 6, 60, 16, 0, 0, 46, 6, 4, 5, 5, 1, 4, 5, 4, 4, 29, 0, 40, 5, 4, 26, 5, 1, 3, 8, 4, 26, 29, 0, 40, 5, 4, 5, 5, 1, 3, 7, 4, 149, 29, 0, 40, 5, 4, 11, 5, 1, 3, 3]
b_y       [344, 58, 172, 4, 701, 0, 2209, 100, 1515, 102, 84, 1, 110, 545, 5, 47, 82, 0, 236, 278, 108, 130, 1615, 1, 963, 557, 83, 502, 63, 0, 1260, 600, 18, 115, 204, 1, 89, 742
, 26, 86, 192, 0, 627, 627, 255, 106, 40, 1, 3, 3]
*****
epoch   0 batch number 137 loss is:  6.186099052429199
prediction [66, 6, 10, 6, 0, 0, 46, 5, 4, 5, 5, 1, 10, 5, 4, 4, 29, 0, 40, 39, 4, 5, 5, 1, 3, 5, 4, 149, 29, 0, 40, 39, 4, 5, 5, 1, 3, 27, 4, 6, 29, 0, 40, 5, 4, 149, 5, 1, 3, 3]
b_y       [649, 134, 430, 220, 1460, 0, 135, 420, 324, 279, 281, 1, 1082, 295, 76, 53, 145, 0, 116, 162, 247, 8, 155, 1, 207, 206, 35, 853, 248, 0, 176, 12, 1017, 13, 455, 1, 1207
, 153, 168, 37, 1169, 0, 829, 469, 10, 14, 307, 1, 3, 3]
*****
```

图 1: 训练过程图

3.2 生成诗歌结果

以下为模型训练完成后，分别以”日、红、山、夜、湖、海、月”为起始字符生成的诗歌。模型采用贪心解码策略，每次选取概率最大的字符：

```
--- 以「山」开头 ---
initial linear weight
山过，乱山仍带霜。
关山不自得，心在不能迟。

--- 以「夜」开头 ---
initial linear weight
波上有风月，风吹上水声。
何当一日月，不见白衣裳。

--- 以「湖」开头 ---
initial linear weight
湖足伴，风光方上土，山水自成丝。
不如无人迹，不见上方还。

--- 以「海」开头 ---
initial linear weight
海海光秋，人间有意不成田。
天生三十年年老，不如一为天上生。

--- 以「月」开头 ---
initial linear weight
月兰木，风吹夜无声。
白日不可得，一时无主人。
何当一日月，不见白头时。
```

图 2: 以不同起始字生成的唐诗结果

可以看出，经过充分训练后，模型能够生成较为符合唐诗格式的文本，相对而言有一定的意境，语义也比较连贯。这表明 LSTM 语言模型对汉语诗歌序列模式的学习也有一定的效果，即使模型结构较为简单，也能实现相应的任务。

4 实验总结

4.1 实验收获

在本次实验中，我通过 PyTorch 实现了基于双层 LSTM 的字符级语言模型，并且能够生成唐诗。为了完成任务，我学习了相关的循环神经网络的基础知识，并进行了实际操作。例如，我对于序列建模的知识有了一定的学习，理解了输入-标签的自回归对齐方式。

4.2 存在的不足

1. **贪心解码的局限性：**贪心解码每步选择概率最高的字符，容易陷入局部最优，导致生成文本重复、缺乏多样性。

2. **诗句意境不足**：模型未引入平仄、押韵等唐诗的特点，导致生成的诗歌在意境方面与古诗有较大的差异。
3. **语义有限**：字符级模型缺乏对于高层语义的理解，同一首诗中前后句的意象衔接有时不够自然。未来可以尝试引入注意力机制（Attention）或基于 Transformer 的预训练语言模型。

4.3 总结

在未来可以从以下几个方面进行改进。首先是将 LSTM 替换为 Transformer 架构，并引入多头自注意力机制，从而更好地建模全局依赖。在解码策略上可以使用 Top-k 采样方法，提升生成的多样性。

通过这次实验，我学习和实现了循环神经网络，验证了它在字符级序列生成任务中的有效性。我了解到双层 LSTM 配合词嵌入和自回归训练范式，能够从大量唐诗数据中学习到汉语诗歌的字符共现规律与语言风格，并在给定起始字后自动续写出具有一定诗意的文本，表明深度学习在自然语言生成领域具有重要作用。今后我也会继续学习相关领域的知识，尝试更多的深度学习模型和方法。